

AI Critique: Digital Wallet Transfer Limits Test Cases

Document Version: 1.0

Author: Solomon Eshiet — QA Engineering

Date: 3 August 2026

Assessment Part: 6A.2 — AI-Assisted Test Design Critique

Inputs Reviewed: Step 6A.1 LLM prompt and raw output (25 cases); product specification (7 rules); pre-AI test plan and top-20 test cases (TC-001–TC-020, coverage matrix M-01–M-15, edge suites E1–E5)

1. What the AI Got Right

Several generated cases are accurate, executable in principle, and align with the stated tier limits and business rules. These represent genuine value from the LLM as a first-pass coverage generator.

Tier limit fidelity. The AI correctly encoded all three tiers: Tier 1 (20,000 per transaction / 50,000 daily), Tier 2 (200,000 / 500,000), and Tier 3 (no per-transaction cap / 5,000,000 daily). Cases TC-AI-001 through TC-AI-003 (happy paths), TC-AI-004 through TC-AI-010 (Tier 1 and Tier 2 boundaries), and TC-AI-024 and TC-AI-025 (Tier 3 daily boundaries) use the correct numeric oracles. This matters because incorrect limit values would produce false passes or false failures across the entire suite.

Core rule coverage. The AI addressed Rule 3 (pending, failure, reversal) in TC-AI-011 through TC-AI-013, Rule 4 (own linked bank exemption) in TC-AI-020 and TC-AI-021, Rule 5 (scheduled execution at 8:00 AM) in TC-AI-015 through TC-AI-017, Rule 6 (immediate tier upgrade) in TC-AI-018 and TC-AI-019, and Rule 7 (rejection with remaining limit) in TC-AI-008, TC-AI-010, and TC-AI-023. The structure — preconditions, steps, expected result — is consistent and readable.

Boundary discipline. TC-AI-006 (20,001 rejected on Tier 1 per-transaction cap), TC-AI-008 (49,000 + 2,000 rejected with remaining limit of 1,000), and TC-AI-010 (499,999 + 2,000 rejected with remaining limit of 1) demonstrate appropriate at-limit, just-under, and just-over thinking. TC-AI-007 (cumulative daily total reaching exactly 50,000) is a useful multi-transfer scenario that a shallow happy-path suite would omit.

Exemption contrast. TC-AI-020 and TC-AI-021 correctly test that transfers to the user's own linked bank account succeed above tier caps, while TC-AI-022 tests that non-exempt bank transfers remain limited. This mirrors my matrix case M-05 (T1 own linked, exempt) and M-04 (T1 third-party bank, limited).

What made these cases good: They translate declarative spec rules into concrete amounts, states, and observable outcomes. They are suitable as draft regression candidates once environment hooks exist. They required minimal correction to the limit arithmetic itself.

2. What the AI Got Wrong or Missed

Despite surface completeness, the AI output contains defects in expected results, spec misreadings, material gaps, redundancy, and implied requirements not present in the specification.

2.1 Incorrect or unreliable expected results

TC-AI-011 (Pending transfer counts toward daily limit) is internally inconsistent. The test submits 25,000 on a Tier 1 account whose per-transaction maximum is 20,000. Under Rules 1 and 7, that transfer should be rejected at submission, never reaching Pending state. The expected result assumes a pending reservation of 25,000 — a state the product should not enter. This is a serious oracle error: the case appears to test pending accounting but actually conflates two failure modes.

TC-AI-019 (Tier upgrade mid-day increases remaining daily limit) hedges in the expected result itself: *"unless product resets usage on upgrade — verify against spec."* The spec states that tier upgrade takes effect immediately (Rule 6) but does not define whether prior daily spend carries forward into the higher cap. The AI produced a numeric oracle (455,000 remaining) without resolving the ambiguity. Presenting a definitive expected result where the spec is silent creates false confidence — the same flaw my test plan flagged in Open Questions 6.1 and daily accounting on upgrade.

TC-AI-014 (Midnight reset) asserts reset behaviour without defining which timezone governs "midnight." The spec does not specify user local time, account timezone, or UTC. The case parenthetically notes *"local timezone as defined by product"* but still writes a pass/fail oracle as if the ambiguity were resolved. My pre-AI design treated midnight reset as **blocked/conditional** (TC-008, E1 suite) until product answered Open Questions 1.1–1.5.

2.2 Misunderstandings and weak spec interpretation

TC-AI-017 (Scheduled transfer rejected when daily limit exceeded at 8:00 AM) covers execution-time rejection (20,000 sent before 8:00 AM; 40,000 scheduled fails) but does not address whether the transfer should have been schedulable at booking time (Open Question 5.2). The AI assumed execution-only validation — one plausible interpretation, not the only one.

TC-AI-022 (Third-party bank account) introduces *"linked but not owned by the user"* verification semantics. Rule 4 exempts the user's **own linked** bank accounts; it does not define a "linked third-party" concept. The scenario is directionally useful but partially hallucinated: linkage ownership rules are not in the spec and belong in security/requirements clarification, not as a definitive functional oracle.

TC-AI-015 expects status progression *"Scheduled to Completed (or Pending then Completed)."* The spec defines execution time only; intermediate statuses are invented. Test oracles should reference defined states or be marked conditional.

2.3 Critical missing scenarios

The AI did **not** generate cases for the highest-risk gaps identified in my pre-AI test plan:

Gap	My pre-AI coverage	AI output
Concurrency / race on remaining daily limit	TC-018; Risk Rank #1	Missing entirely
Pending transfer straddling midnight	TC-009; E1 suite; Risk Rank #2	Missing — only simple midnight reset (TC-AI-014)
Tier upgrade while transfer is in-flight	TC-015; Risk Rank #5	Partial — TC-AI-018 covers post-upgrade new transfer, not mid-flight re-evaluation
Timezone / DST for midnight and 8:00 AM	Open Questions 1.1–1.5; E1	Missing
Scheduled transfer when limit exhausted — booking vs execution	TC-016, TC-017; Open Question 5.1	Partial — TC-AI-017 only

Tier 2 per-transaction boundary (200,001)	M-07	Missing — Tier 2 coverage is daily-only
Wallet-to-bank vs wallet-to-wallet matrix	M-01–M-15 pairwise matrix	Implicit — most cases say "wallet user" only
Reversal credited after daily reset	E3 / Open Question 4.2	Missing
Idempotent retry / duplicate submission	E2 / Open Question 3.4	Missing

The AI produced breadth across **static limits** but missed **temporal interaction** and **concurrency** — precisely where my risk register ranked financial and compliance exposure highest.

2.4 Redundancy and low-value duplication

- **TC-AI-004 and TC-AI-005** both validate sub-cap Tier 1 per-transaction behaviour (20,000 vs 19,999). TC-AI-005 adds negligible information once TC-AI-004 and TC-AI-006 exist.
- **TC-AI-008, TC-AI-010, TC-AI-023, and TC-AI-025** repeatedly exercise Rule 7 error messaging with different tiers. One parametrized case per tier would suffice; four near-duplicates inflate suite size without proportional risk reduction.
- **TC-AI-001 through TC-AI-003** are structurally identical happy paths scaled by tier — acceptable for onboarding, but my equivalence-partitioning approach (M-01, M-08, M-09) treated "under limit" as one class per tier with spot checks rather than three full procedural duplicates.

2.5 Hallucinated or unstated requirements

The AI implied: defined Pending vs Completed state machine (TC-AI-015); notification on scheduled failure (TC-AI-017); linked-account ownership verification model (TC-AI-022); and timezone resolution (TC-AI-014). None appear in the seven-rule spec. These are reasonable **questions for product**, not acceptable as **test oracles** without explicit confirmation.

3. What I Kept vs What I Discarded (and Why)

AI Case	Decision	Rationale
TC-AI-003	Kept (maps to M-09 / TC-003)	Correctly tests Tier 3 large single transfer with no per-transaction cap — unique tier behaviour.
TC-AI-006	Kept (maps to M-03)	Valid per-transaction over-limit rejection at 20,001 for Tier 1.
TC-AI-007	Kept (maps to M-02 daily leg)	Cumulative daily boundary at exactly 50,000 across two transfers.
TC-AI-008	Kept (maps to M-14)	Distinguishes daily vs per-transaction rejection; aligns with Rule 7 remaining-limit message.
TC-AI-012	Kept (maps to TC-011 failure path)	Failure restores daily limit — core Rule 3 behaviour.
TC-AI-013	Kept (maps to TC-012)	Reversal restores limit — core Rule 3 behaviour.

TC-AI-016	Kept (maps to TC-017 partial)	Scheduled transfer counts on execution day — necessary scheduling/limit interaction.
TC-AI-018	Kept (maps to tier upgrade happy path)	Immediate upgrade enables higher per-transaction amount — valid Rule 6 test.
TC-AI-020	Kept (maps to M-05)	Exempt path above Tier 1 per-transaction cap — high-value Rule 4 case.
TC-AI-021	Kept (maps to M-05 / exempt daily)	Exempt when daily wallet limit exhausted — confirms exemption is total.
TC-AI-024	Kept (maps to M-10)	Tier 3 daily maximum in one transfer — boundary at 5,000,000.
TC-AI-011	Discarded	Invalid precondition: 25,000 exceeds Tier 1 per-transaction limit; pending state unreachable.
TC-AI-014	Discarded as written	Retained intent in my TC-008 but marked blocked pending timezone answer; AI oracle overstates spec certainty.
TC-AI-015	Discarded	Replaced by my TC-016 with clock-injection requirement; AI version relies on wall-clock wait and undefined statuses.
TC-AI-017	Discarded as sole schedule-failure case	Partially merged into TC-016/TC-017 suite; insufficient alone and ignores booking-time validation.
TC-AI-019	Discarded	Non-definitive oracle; retained upgrade scenario only after product clarifies daily carry-forward (Open Q 6.1).
TC-AI-005	Discarded	Redundant with TC-AI-004 and TC-AI-006.
TC-AI-022	Discarded as written	Replaced by M-04 (third-party bank, limited) without inventing "linked not owned" verification.
TC-AI-023	Discarded	Redundant Rule 7 message test; retained one parametrized error-contract case in automation layer.
TC-AI-025	Discarded	Redundant with TC-AI-024 and TC-AI-010 pattern; daily over-limit already covered.
<i>(not generated)</i>	Retained from my suite	TC-018 concurrency race, TC-009 pending-at-midnight, TC-015 tier upgrade in-flight — AI omitted these entirely.

Net result: I incorporated approximately **11 of 25** AI cases after correction or mapping, rejected **9** for incorrect oracles, redundancy, or hallucination, and **supplemented with 9+ scenarios** the AI never produced. The final executable suite is a hybrid: AI drafts for static boundaries, human design for interaction risk.

4. What This Exercise Reveals About AI in Test Design

4.1 Where AI helps

Large language models demonstrate clear strength in **speed and breadth** for rule-based features. Given a concise spec, the AI produced 25 structured cases in minutes, covering all three tiers, basic boundaries, pending/failure/reversal, scheduling, exemption, and upgrade — a useful **brainstorming draft** that would take a human significantly longer to type from scratch. For onboarding junior QA staff or populating an initial test-case skeleton before review, this is genuinely productive.

AI also performs well on **deterministic arithmetic oracles** where the spec is unambiguous: reject 20,001 on Tier 1, show remaining limit of 1,000 when 49,000 is already sent. Parameterised templates (tier x amount x expected outcome) are natural LLM outputs and map cleanly to automated API tests.

4.2 Where AI harms

The same output creates **false confidence**. Twenty-five cases look comprehensive; my review found one internally invalid case (TC-AI-011), several non-definitive oracles, and absence of the five highest-risk scenarios from my register. A team that executed the AI suite green without critique could ship with undetected concurrency and midnight-straddle defects — failures with direct financial-control implications.

AI lacks **domain judgment** and **requirements gatekeeping**. It did not produce an open-questions log, did not flag blocked tests, and filled spec silence with plausible behaviour (timezone, status names, notifications). Senior QA work is largely deciding what **cannot** be tested yet — the AI defaulted to testing everything as if the spec were complete.

AI also **optimises for volume over risk**. Four variations of "show remaining limit on reject" and duplicate sub-boundary cases suggest pattern completion rather than risk-based prioritisation. My 70/20/10 automation split and pairwise matrix explicitly avoided this exhaustiveness trap.

4.3 Personal policy for AI in test design going forward

I will use AI as a **draft generator and review checklist**, never as an authoritative test oracle or submission-ready deliverable. My policy:

1. **Human owns the spec critique first.** Open questions, risk register, and blocked oracles are written before or independently of AI output — as I did in Part 1 — so AI cannot anchor the team to incomplete requirements.
2. **AI output is mandatory peer review.** Every case is validated for precondition feasibility, numeric correctness, spec traceability, and duplication before entering the repository. TC-AI-011 would be caught in a five-minute review.
3. **AI fills the matrix; humans own the edges.** I will prompt for happy paths and boundaries, then manually add concurrency, time boundaries, state-machine transitions, and fraud paths — the defects AI consistently misses.
4. **No AI-only CI oracles for financial controls.** Limit enforcement tests that gate releases require human-signed expected results tied to confirmed product answers. AI may suggest wording; it does not sign off.
5. **Document the hybrid explicitly.** Assessment and audit trails should state which cases are AI-drafted vs human-designed, preserving accountability.

Used this way, AI compresses documentation time without replacing the judgment that distinguishes a checkbox exercise from production-ready quality engineering.

Document Control

Version	Date	Author	Changes
1.0	3 Aug 2026	Solomon Eshiet	Initial submission for Part 6A.2

Related documents: AI-Generated-Test-Cases-Digital-Wallet-Raw.md (Step 6A.1); Test-Plan-Digital-Wallet-Transfer-Limits.md (Part 1); Top 20 Test Cases TC-001-TC-020 (Part 1.3)

End of AI Critique Document